



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2333 — Sistemas Operativos y Redes — 1/2026 Tarea 2 - Memoria

Miércoles 29 de Abril 2026

Fecha de Entrega: Viernes 15 de Mayo a las 22:00 hrs.

Composición: Tarea en parejas

Objetivo

- Construir una **biblioteca (API)** que permita leer y escribir datos en una **memoria principal simulada**, cuidando la asignación y liberación de memoria.

Homer Memory



Figura 1: Jonathan Frink y Homero.

Homero Simpson ha comenzado a olvidar absolutamente todo: el nombre de sus hijos, dónde vive, e incluso cómo abrir una lata de Duff. Desesperada, Marge lleva a Homero con el Profesor Jonathan Frink, el brillante científico de Springfield. Frink, tras varios “glavin” y ajustes a sus lentes, propone una solución revolucionaria: digitalizar y resguardar la memoria de Homero en un archivo binario altamente estructurado... Mientras Frink digitaliza la memoria de homero, tú, como ayudante de Frink, deberás encargarte de **programar una API que le permita trabajar con la información dentro de ella.**

Frink decide trabajar la memoria de Homero como un archivo binario, el cual estará dividido en secciones¹ y en *frames* de 32 KB. La API debe crear/terminar procesos simulados, asignar y liberar *frames*, y administrar archivos que viven dentro del espacio de direcciones virtual de cada proceso (128 MB paginados). Cada operación sobre un archivo usa una dirección **virtual** ($VPN + offset$) que se traduce a una dirección **física** ($PFN + offset$) consultando la **tabla de páginas invertida**. Toda acción debe mantener consistentes la **tabla de PCBs**, la **Tabla Invertida de Páginas (IPT)** y el **frame bitmap**, de modo que el estado de la memoria simulada quede correctamente reflejado en la memoria utilizada (el archivo `.bin`). Para que puedas avanzar mientras Frink trabaja en el respaldo, te entrega una memoria distinta con la misma estructura.

Introducción breve a paginación

Paginación divide la memoria virtual de un proceso en bloques de igual tamaño llamados **páginas**. La memoria física se divide en bloques del mismo tamaño llamados **frames**.

Idea clave: una **página** del proceso puede estar almacenada en **cualquier frame** físico. Así evitamos la **fragmentación externa**.

En este proyecto trabajarás con una versión *simplificada* de paginación sobre un archivo real (la memoria simulada). Para repasar:

- [Paginación \(video\)](#)
- [Tabla de páginas invertida \(video\)](#)
- [Segmentación \(video\)](#)

Estructura de la memoria `homer_memory`

La memoria física es un archivo con 4 zonas, en este orden:

Sección	Tamaño
Tabla de PCBs	8 KB
Tabla invertida de páginas	192 KB
Frame Bitmap	8 KB
Área de Datos	2 GB

Cada frame dentro de los **2 GB** posee un tamaño de **32 KB**. Hay en total 2^{16} frames. Cada frame tiene un número (**PFN**) que cabe en **2 bytes** (se recomienda `uint16_t`).

Cada proceso contará con un espacio virtual de memoria de **128 MB**, esta memoria está dividida en páginas de **32 KB**.

Ayuda práctica: para una dirección virtual con $(VPN, offset)$ y su PFN, la dirección física **relativa** a la zona de frames es:

$$paddr_rel = (PFN \ll 15) \text{ or } offset$$

² y la **absoluta** (en el archivo) es:

$$paddr_abs = |PCBs| + |IPT| + |Bitmap| + paddr_rel.$$

¹ estas secciones están detalladas por la estructura de la memoria, indicada más adelante

² El $PFN \ll 15$ nace por los 32 KB ($= 2^{15}$ bytes)

Tabla de PCBs (procesos)

32 entradas, cada una con **256 bytes**, en el **siguiente orden exacto**:

- **1 byte de estado**: 0x01 existe, 0x00 libre.
- **14 bytes nombre del proceso**. Nota: se almacenan hasta 14 bytes sin '\0'. Al copiarlos a buffers locales, usar 15 bytes y forzar `name[14]='\0'`.
- **1 byte id** del proceso.
- **240 bytes** para su **Tabla de Archivos**.

Tabla de Archivos (por proceso)

Hasta **10** archivos, cada entrada:

- **1 byte** de validez (0x01 válido, 0x00 vacío).
- **14 bytes** nombre del archivo (sin '\0', al copiar, use 15B y `name[14]='\0'`).
- **5 bytes tamaño de archivo** como entero **sin signo de 40 bits** en **little endian**.
- **4 bytes dirección virtual** con este formato³:



Tabla Invertida de Páginas

Esta tabla traduce (**pid, VPN**) → **PFN**. La **dirección física** que obtengas es **relativa** al bloque final de 2 GB. La **dirección absoluta** está dada por:

$$2^{13} + 3 \cdot 2^{16} + 2^{13} + \text{dir}$$

(es decir, `|PCBs| + |IPT| + |Bitmap| + dir`).

Estructura:

- **24 bits** por entrada (leer como **3 bytes contiguos** y extraer bytes en **little endian**).
- **65536** entradas (una por **PFN**, en orden).
- **1 bit** de validez (1 válido, 0 inválido).
- **10 bits** para *pid* (los **2 primeros no significativos**).
- **13 bits** para **VPN** (el **primer bit no significativo**).

Traducción virtual → física:

1. Extraer **12 bits** de **VPN** y **15 bits** de **offset** desde la **dirección virtual**.
2. Buscar en la Tabla invertida de páginas la entrada (**pid, VPN**), el índice es el **PFN**.
3. La **dirección física relativa** es **PFN** concatenado con **offset**.

³ Los primeros 5 bits son no significativos

Frame bitmap

Tamaño **8 KB = 65536 bits**. Cada bit representa un frame: **0** libre, **1** ocupado.

- Hay un bit por cada frame de la memoria física.
- Debe reflejar siempre el **estado real** de la memoria.

Frames (datos)

Aquí viven los contenidos de los archivos. Cada frame mide **32 KB**. Están en los últimos **2 GB**. Total: 2^{16} **frames**.

Endianness (orden de bytes)

Leer y escribir en **little endian**. Esto importa cuando el dato ocupa **más de 1 byte** (p.ej., tamaños y direcciones). Usando tipos correctos, normalmente no tendrás que *swappear* manualmente.

API de homer memory

Implementa una biblioteca en `homer_memory_API.c` con su *header* `homer_memory_API.h`. Define además un `struct homerFile`⁴ para representar un archivo abierto.

Para probar, crea un `main.c` que incluya `homer_memory_API.h` y use las funciones sobre un archivo de memoria (archivo de la forma `.bin`) pasado por línea de comandos.

Funciones generales

- `void mount_memory(char* memory_path)` : Función para montar la memoria. Establece como variable global la ruta local donde se encuentra el archivo `.bin` correspondiente a la memoria.
- `void list_processes()` : Imprime información de cada uno de los procesos activos, de la forma:

```
<PROCESS_ID> <PROCESS_NAME>
```

- `int processes_slots()` : Retorna la **cantidad de entradas libres** de la tabla de PCBs.
- `void list_files(int process_id)` : Recibe un id de un proceso, e imprime información de cada archivo dentro de la memoria virtual de este, de la forma:

```
<VPN> <FILE_SIZE> <DIRECCION_VIRTUAL> <FILE_NAME>.
```

Tanto el VPN como la dirección virtual deben estar en formato hexadecimal, `file_size` en decimal y `file_name` como string.

- `void frame_bitmap_status()` Imprime la cantidad de frames usados y los libres, con el formato:

```
USADOS: <N-USADOS> LIBRES: <N-LIBRES>
```

⁴ Usa `typedef` para renombrarlo.

- `int format_memory(char* memory_path)` : La función debe abrir (o crear) el archivo indicado en la ruta, dejándolo en un formato que permita trabajar sobre esta memoria (es válido dejar todo vacío). **Retorna 0** en caso de que la memoria haya sido inicializada correctamente, y **retorna -1** en cualquier otro caso.

Funciones para procesos

- `int start_process(int process_id, char* process_name)` : Crea un proceso con el `process_id` y el `process_name`. Se debe guardar su información respectiva en la tabla de PCBs. Cualquier cambio realizado en este proceso, debe reflejarse en la memoria montada. Si el proceso fué iniciado correctamente **retorna 0**, en cualquier otro caso **retorna -1**.
- `int finish_process(int process_id)` : Recibe un `process_id` (id de proceso), y termina el proceso correspondiente, liberando su memoria y borrando su entrada válida en la **Tabla de PCBs**. **Retorna 0** si termina de manera correcta, en cualquier otro caso **retorna -1**.
- `int clear_all_processes()` : Termina todos los procesos activos, liberando su memoria y sus entradas válidas en la tabla de PCBs. **Retorna la cantidad de procesos terminados**.
- `int file_table_slots(int process_id)` : Recibe un `process_id`, y **retorna** la cantidad entradas **libres** dentro de su tabla de archivos.

Funciones para archivos

- `homerFile* open_file(int process_id, char* file_name, char mode)` : recibe un modo (mode), que puede ser 'r' o 'w':
 - Si el modo es 'r', entonces busca el archivo con el nombre recibido y **retorna el homerFile*** que lo representa. Si no lo encuentra **retorna NULL**.
 - Si el modo es 'w', entonces se verifica que el archivo no exista y se **retorna el homerFile*** con el id y nombre recibidos. Si el archivo existe **retorna NULL**.
- `int read_file(homerFile* file_desc, char* dest)` : Lee el archivo (`file_desc`) y lo copia a `dest` en el sistema local. Retorna bytes leídos. *Ojo:* si el archivo ocupa varias páginas/frames, continúa secuencialmente.
- `int write_file(homerFile* file_desc, char* src)` : Escribe el archivo de `src` dentro de la memoria montada. Este archivo, además, debe ser reflejado en el `file_desc`. **Retorna la cantidad de bytes escritos**. La escritura debe iniciar desde el primer espacio libre en la memoria virtual **no necesariamente se escribirá desde un inicio de página**. Por lo tanto, los archivos **pueden compartir tanto frame como página**. La escritura de archivo debe detenerse cuando no queden frames disponibles o cuando se termine el espacio contíguo en la memoria virtual.
- `void delete_file(int process_id, char* file_name)` : Quita el archivo de la memoria virtual del proceso. Si un frame queda **totalmente libre**, marca 0 en el bitmap e **invalida** su entrada en la tabla invertida de páginas.
- `void close_file(homerFile* file_desc)` : Cierra el archivo indicado por `file_desc`.

Bonus

- `void memory_report(char* output_path) :` Crea un reporte en `output_path`. El reporte corresponde a el estado de toda la memoria, conteniendo `process_slots` y `frame_bitmap_status`, seguido por `list_files` y el porcentaje de entradas libres para cada proceso, separados por una línea en blanco.

```
SLOTS: <process_slots>
FRAME.USADOS: <N.USADOS> FRAME.LIBRES: <N.LIBRES>
PROCESO <PID>:
<VPN> <FILE.SIZE> <DIRECCION.VIRTUAL>
...
<VPN> <FILE.SIZE> <DIRECCION.VIRTUAL>
USAGE: <porcentaje de entradas libres>%
//Línea en blanco
...
//Línea en blanco
PROCESO <PID>:
<VPN> <FILE.SIZE> <DIRECCION.VIRTUAL>
...
<VPN> <FILE.SIZE> <DIRECCION.VIRTUAL>
USAGE: <porcentaje de entradas libres>%
```

Ejecución

Tu programa principal debe llamarse `homer_memory` y recibir un archivo binario que simule memoria, por ejemplo:

```
./homer_memory mem.bin
```

Ejemplo de uso dentro de `main.c`:

```
mount_memory(argv[1]); // Monta memoria
start_process(10, "ventas"); // Crea proceso 10
start_process(20, "reportes"); // Crea proceso 20
list_processes(); // Lista procesos
int pcb_libres = processes_slots(); // PCB slots libres
printf("PCB free slots: %d\n", pcb_libres);
int slots_arch = file_table_slots(10); // Slots archivo libres
printf("File-table free slots (pid=10): %d\n", slots_arch);
homerFile* f = file_open(10, "log.txt", 'w'); // Abrir/crear archivo (w)
write_file(f, "log_local.txt"); // Escribir desde local
close_file(f); // Cierra descriptor
f = file_open(10, "log.txt", 'r'); // Reabrir lectura
read_file(f, "log_copia.txt"); // Copiar a local
close_file(f); // Cierra descriptor
list_files(10); // Listar archivos (hex)
frame_bitmap_status(); // Bitmap: usados/libres
delete_file(10, "log.txt"); // Eliminar archivo p10
finish_process(10); // Terminar proceso 10
int cerrados = clear_all_processes(); // Cerrar todos
printf("Procesos cerrados por clear_all_processes(): %d\n", cerrados);
```

Puedes aplicar cada operación directamente sobre `mem.bin` o acumular cambios en memoria y volcar al final. Lo importante es que **el archivo refleje el estado final correcto**.

Para probar se entregarán en Canvas:

- `memformat.bin`: memoria vacía (sin procesos ni archivos).
- `memfilled.bin`: memoria con procesos y archivos ya escritos.

Observaciones útiles

- Siempre parte con **montar** la memoria.
- Un archivo puede **no estar** en frames contiguos (es normal en paginación).
- **No** es requisito **defragmentar**, se permite **fragmentación interna**.
- Si una entrada tiene **bit de validez/estado**, basta con ponerlo en 0 para invalidar.
- Si se llena el espacio durante `write_file`, **detén** la escritura. **No borres** lo ya escrito.
- Cualquier cosa no especificada: puedes hacer **supuestos razonables** y listarlos en el README.

Formalidades

Sube sólo el **código fuente** necesario y un **Makefile**.

- Trabajo **en parejas**.
- Lenguaje: **C**. Otros lenguajes no serán revisados.
- **NO** incluyas **binarios**⁵. Penaliza **0.2 puntos**.
- **NO** incluyas el **repo git**⁶. Penaliza **0.2 puntos**.
- Grupo mal inscrito: **-0.3 puntos**.
- Debe compilar con `make` y generar `homer_memory`. Sin **Makefile**: **-0.5 puntos** y riesgo de no corrección.
- Formato de entrega incorrecto: **-0.3 puntos**.
- Si **no compila** o produce **segmentation fault**, nota mínima. Se permite recorregir cambiando líneas con **-0.1 por cada 4 líneas** (máx. 20 líneas).

Código desordenado puede tener descuentos extra a criterio del ayudante. **Modulariza**, usa **funciones** y **nombres claros**. Entregas fuera de la carpeta pedida **no se corrigen**.

⁵ Ejecutables, `.o`, etc.

⁶ Si lo hiciste, borra `.git` antes de entregar.

Evaluación

6.0 pts Funciones de biblioteca:

- 0.2 mount_memory
- 0.3 list_processes
- 0.2 processes_slots
- 0.5 list_files
- 0.5 frame_bitmap_status
- 0.3 format_memory
- 0.3 start_process
- 0.4 finish_process
- 0.4 clear_all_processes
- 0.2 file_table_slots
- 0.3 open_file
- 0.6 read_file
- 0.8 write_file
- 0.8 delete_file
- 0.2 close_file

+0.5 pt (libres) memory_report.

Descuento por memoria: hasta **-1.0 pt** si valgrind reporta leaks de memoria.

Entrega

Dentro de esta entrega es importante que:

1. Entreguen uno o más **scripts main.c** que **muestren** el uso de **todas** las funciones que implementaron. Si no se evidencia durante la corrección, **no se evalúa**.
2. Suban los `main.c` y los archivos que estimen necesarios.
3. En la corrección se descargarán sus fuentes y se ejecutará con su `main.c` u otro hecho por los ayudantes (la API no debe funcionar solo para sus `main.c`).
4. La tarea debe ser subida al servidor en la carpeta T2.
5. Pueden usar archivos de cualquier tipo (p.ej., **gif/audio**) para mostrar límites de tamaño.
6. Pueden usar **más de un** `main.c` si ayuda a mostrar funcionalidades distintas.

Política de atraso

Hasta **2 días de atraso**:

$$N_{T2}^{\text{Atraso}} = \min(N_{T2}, 7,0 - 0,75 \cdot d)$$

d es días de atraso. Es un **descuento soft**: baja la **nota máxima alcanzable**. No transfiere días a otras tareas.

Preguntas

Dudas en el [foro oficial](#). **48 horas antes** del cierre se dejan de responder preguntas.